

Chapitre 4: Pointeurs et structures de données

Dr. Khadhraoui Mayara

2025/2026

PLAN

1. Les pointeurs
2. Les structures de données

Section 1

Les pointeurs

- Mémoire d'un ordinateur $\approx$ ensemble de **cases-mémoire**. Chaque case est repérée par une **adresse**.

	Mémoire	Adresse
		0
		1
		2
		:
var	12	66844 (= &var)
		:

- L'adresse où est stockée la valeur d'une variable `var` s'obtient grâce à l'opérateur d'adresse `&` : cette adresse se note `&var`.

- **Pointeur** : variable contenant l'adresse d'une autre variable. La déclaration d'un pointeur est de la forme suivante :

- `<type> *<nom>;`

Par exemple :

```
int *p; // pointe sur une case quelconque
```

```
int x = 0;
```

```
cout << x << endl; // affiche 0
```

	Mém.	Adr.
p	58456	: 18424
x	0	: 36758
*p		: 58456

La variable p est alors un pointeur sur un int. La variable entière dont p contient l'adresse est dite pointée par p et se note *p.

- **Pointeur** : variable contenant l'adresse d'une autre variable. La déclaration d'un pointeur est de la forme suivante :

- `<type> *<nom>;`

Par exemple :

```
int *p; // pointe sur une case quelconque
```

```
int x = 0;
```

```
p = &x; // pointe maintenant sur x
```

```
cout << x << endl; // affiche 0
```

	Mém.	Adr.
p	36758	: 18424
*p,x	0	: 36758
		: 58456

La variable `p` est alors un pointeur sur un `int`. La variable entière dont `p` contient l'adresse est dite pointée par `p` et se note `*p`.

- **Pointeur** : variable contenant l'adresse d'une autre variable. La déclaration d'un pointeur est de la forme suivante :

- `<type> *<nom>;`

Par exemple :

`int *p; // Déclaration d'un pointeur vers un entier (il peut contenir l'adresse d'un int)`

`int x = 0; // Déclaration d'une variable entière x, initialisée à 0`

`p = &x; // Le pointeur p reçoit l'adresse de x → p pointe maintenant vers x`

`*p = 5; // On accède à la case mémoire pointée par p (donc x) et on y met la valeur 5`

`cout << x << endl; // Affiche la valeur de x (qui vaut maintenant 5)
puis passe à la ligne`

	Mém.	Adr.
p	36758	: 18424
*p,x	5	: 36758
		: 58456

int *p;

- Déclare une **variable pointeur** p qui peut contenir l'adresse d'un int.
- **Type** : p est de type "pointeur vers int".
- Important : ici p **n'est pas initialisé** — il contient une adresse indéterminée (valeur aléatoire). Si on l'utilisait (*p) avant de l'initialiser, on risque un comportement indéfini (plantage).
- Conseil sûr : `int *p = nullptr;` pour l'initialiser proprement.

int x = 0;

- Crée une variable entière x et lui donne la valeur 0.
- En mémoire : une case contenant 0.
- `p = &x;`
- L'opérateur & signifie "adresse de". &x donne l'adresse mémoire où x est stocké.

2. Les pointeurs

- On affecte cette adresse à p. Maintenant p **pointe vers** x (c'est-à-dire qu'il contient l'adresse de x).
- Schéma mental : p ---> [case mémoire de x] (contient 0)

***p = 5;**

- * est l'opérateur de **déréférencement** : il accède à la valeur stockée à l'adresse contenue dans p.
- *p = 5; signifie donc : « aller à l'adresse contenue dans p (ici l'adresse de x) et y écrire la valeur 5 ».
- Effet : **la variable x devient 5.**

cout << x << endl;

- Affiche la valeur actuelle de x sur la console, puis passe à la ligne.
- Comme *p = 5 a modifié x, l'affichage sera 5.

Initialisation

Un pointeur s'initialise à nullptr.

```
double *ptr = nullptr; // aucune case mémoire n'est pointée
```

Types

Un pointeur est toujours lié au type sur lequel il pointe.

```
int *ip;
```

```
double *dp;
```

```
dp = ip; // illégal car ip et dp ne sont pas de même type
```

```
*dp = *ip; // correct (conversion implicite de type)
```

3. Les références

- Une **référence** est une variable qui coïncide avec une autre variable. La déclaration d'une référence est de la forme suivante :

<type> &<nom> = <nom_var>;

- Par exemple :

```
int n = 10;  
int &r = n; // r est une référence/alias sur n
```

n et r ont la **même adresse** : r peut être vue comme un alias de n.

```
r = 20; // modification de la valeur de n
```

- **Contrairement aux pointeurs**, une référence ne peut être initialisée qu'une seule fois : à la déclaration.
- Toute autre affectation modifie en fait la variable référencée.

3. Les pointeurs vs les références

Caractéristique	Pointeur (int *p)	Référence (int &r)
Contenu	Contient l'adresse d'une variable	Alias : pas une variable à part, juste un autre nom pour une variable existante
Accès à la valeur	*p	r directement
Déclaration	int *p = &x;	int &r = x;
Peut être nul	✓ Oui (p = nullptr;)	✗ Non, une référence doit toujours être liée à une variable
Peut changer de cible	✓ Oui (p = &y;)	✗ Non, r reste toujours lié à la même variable
Besoin d'opérateur *	✓ Oui pour accéder à la valeur	✗ Non, on utilise r directement
Allocation mémoire	Le pointeur est une variable indépendante	Pas de nouvelle case mémoire, partage la même que la variable originale
Modification de la valeur originale	✓ *p = 20;	✓ r = 20;

4. Passage des paramètres dans une fonction

Version naïve (passage par valeur) :

```
void swap(int a, int b) // ne fonctionne pas
{
    int aux; // variable auxiliaire servant pour l'échange
    aux = a;
    a = b;
    b = aux;
}
```

- Les lignes `aux = a; a = b; b = aux;` **échangent les copies a et b, pas x et y eux-mêmes.**
- Une fois que la fonction se termine, ces **copies disparaissent**, donc **x et y restent inchangés.**
- En C/C++, quand on passe un `int` à une fonction **par valeur**, la fonction ne reçoit qu'une **copie** de la variable.
- Tout changement sur cette copie **n'affecte pas la variable originale.**

4. Passage des paramètres dans une fonction

Version par références :

```
void swap(int &a, int &b) // style C++
{
    int aux; // variable auxiliaire servant pour l'échange
    aux = a;
    a = b;
    b = aux;
}
```

- **int &a** et **int &b** → a et b sont des alias des variables originales, donc tout changement affecte directement les variables appelantes.
 - **aux** sert juste à ne pas perdre la valeur de a pendant l'échange.
 - La fonction modifie **réellement les valeurs originales**, contrairement à la version “naïve” qui utilisait le passage par valeur.
- ➔ **Par référence** → échange directement les variables, simple et sûr.

4. Passage des paramètres dans une fonction

Version par pointeurs :

```
void swap(int *a, int *b) // style C
{
    int aux; // variable auxiliaire servant pour l'échange
    aux = *a;
    *a = *b;
    *b = aux;
}
```

- `int *a, *b` → `a` et `b` **contiennent des adresses** de variables, pas les valeurs elles-mêmes.
- `*a` et `*b` → permettent **d'accéder aux valeurs stockées aux adresses pointées**.
- L'échange fonctionne sur les **valeurs originales**, car on manipule **directement les adresses des variables**.

➔ **Par pointeur** → échange via les adresses, plus flexible mais plus compliqué à écrire.

&, quand utilisé avec une déclaration de variable (un type)

= **référence sur...**

```
int &ra = a; // ra est une référence sur a (un alias de a)
```

*, quand utilisé avec une déclaration de variable (un type)

= **pointeur sur...**

```
int *pa; // pa est un pointeur sur un entier
```

&, quand utilisé avec une variable déjà déclarée

= **adresse de...**

```
&a // adresse de la variable a
```

*, quand utilisé avec un pointeur déjà déclaré

= **valeur pointée par...**

```
cout << *pa << endl; // affiche la valeur pointée par pa
```

On dit aussi que * est un opérateur de **déréférencement**.

En C/C++ :

Le nom du tableau **T** n'est pas vraiment une variable ordinaire, il est un **pointeur constant vers le premier élément** du tableau.

Autrement dit :

$T \equiv \&T[0]$

T contient **l'adresse mémoire du premier élément** (T[0]).

Considérons :

```
int T[5]; // T est un tableau de 5 entiers
```

```
int *p = T;
```

Passage de tableau en paramètre de fonctions:

Il se fait par valeur, et :

- il n'y a pas de copie du contenu du tableau (gain de temps)
- la fonction peut modifier le contenu du tableau

```
void annule(int *V, int n) // notation équivalente
// Déclare une fonction appelée annule
// int *V = pointeur vers le premier élément d'un tableau d'entiers
// int n = taille du tableau
// void = la fonction ne retourne rien
{
    for(int i = 0; i < n; i++) // boucle pour parcourir tous les indices de 0 à n-1
        V[i] = 0;             // met à 0 chaque élément du tableau
}
```

8. Variables dynamiques / statiques

Supposons déclaré :

```
int *p; // pointeur sur int
```

Pour l'instant, p ne pointe sur rien. Il faut réserver l'espace-mémoire nécessaire à un `int` par l'instruction:

```
p = new int; // new : opérateur d'allocation mémoire
```

Cette fois, p contient une véritable adresse, décidée par la machine pendant l'exécution.

On peut alors utiliser la variable pointée par p (qu'on nomme **variable dynamique** par opposition aux variables ordinaires qualifiées de **statiques**) :

```
*p = 12;
```

La variable pointée par `p` n'est pas automatiquement désallouée en sortie de bloc (c'est l'intérêt). Lorsqu'on n'a plus besoin de la variable `*p`, il faut libérer l'espace-mémoire qu'elle occupe par l'instruction :

`delete p;` // delete : opérateur de désallocation mémoire

L'expression `*p` est dès lors illégale, jusqu'à une prochaine allocation par :

`p = new int;`

9. Gestion de la mémoire- variables dynamiques

Grâce aux pointeurs, on peut manipuler des **structures de données dynamiques** dont la taille n'est déterminée qu'au moment de l'exécution.

Supposons déclaré :

```
int *T;
```

L'allocation-mémoire nécessaire pour recevoir n entiers s'écrira :

```
T = new int[n]; // un tableau dynamique de n entiers
```

Intérêt : n n'est pas nécessairement une constante. La taille du tableau peut donc être décidée au moment de l'exécution, en fonction des besoins.

Désallocation :

```
delete [] T; // désallocation mémoire d'un tableau dynamique
```

10. Comprendre les pointeurs: exemple du double tableau

Une matrice 2D (double tableau) peut s'allouer dynamiquement. Il s'agit d'un tableau 1D où chaque case pointe vers un autre tableau 1D : **c'est un tableau de tableaux.**

On l'implémente comme un pointeur de pointeurs, donc :

```
int **T; // déclaration du double pointeur,  
        // il pointera sur le tableau 2D d'entiers
```

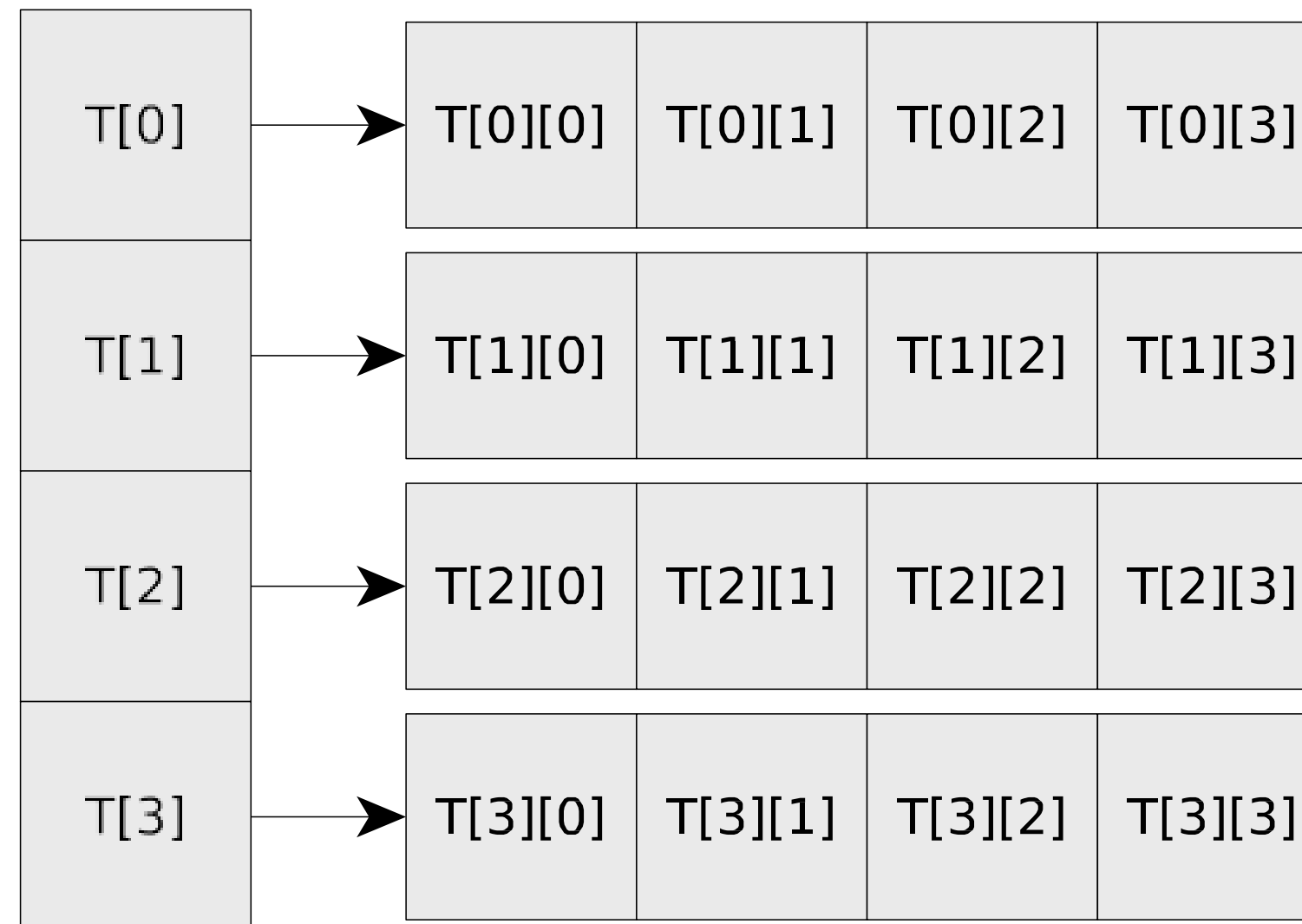
L'allocation-mémoire se déroule en deux temps :

```
T = new int*[nb_row]; // allocation du premier tableau : c'est un tableau de pointeurs sur  
des entiers (i.e. un tableau de tableaux d'entiers) ; le type de chaque cellule du tableau  
sera donc int*  
for(int i = 0; i < nb_row; ++i)  
    T[i] = new int[nb_col]; // allocation d'un tableau de nb_col éléments dans chaque case  
du premier tableau. Chaque élément de ce sous-tableau est de type int
```

Illustration de l'allocation dynamique :

```
int **T = new int*[nb_row];
```

```
for(int i = 0; i < nb_row; ++i)  
    T[i] = new int[nb_col];
```



Section 2

Les structures de données

1. Structures de données Séquentielles et linéaires

- Structures de données séquentielles (Tableaux)
- Structures de données linéaires (listes chaînées, piles, files)

Définition

*Un **tableau** est une structure de donnée T qui permet de stocker un certain nombre d'éléments $T[i]$ repérés par un index i . Les tableaux vérifient généralement les propriétés suivantes :*

- *tous les éléments ont le même type de base;*
- *le nombre d'éléments stockés est fixé;*
- *l'accès et la modification de l'élément numéro i est en temps constant $\Theta(1)$, indépendant de i et du nombre d'éléments, le tableau.*

NB: $\Theta(1)$: Cela signifie que **peu importe la position de l'élément dans le tableau (début, milieu ou fin)**, le temps nécessaire pour **accéder** à cet élément ou le **modifier** est **le même**.

Un tableau en mémoire

Définition

Dans le tableau, tous les éléments ont la même taille mémoire.

Nombre d'élément : n , taille d'un élément t :

$T[0]$	$T[1]$	$T[2]$	$\dots$	$T[n-1]$
--------	--------	--------	---------	----------

On suppose que le tableau commence à l'adresse d

- $T[0]$ occupe les cases d à $d + t - 1$;
- $T[1]$ occupe les cases $d + t$ à $d + 2t - 1$;
- $T[i]$ occupe les cases $d + it$ à $d + (i + 1)t - 1$;
- Le tableau entier occupe les cases d à $d + nt - 1$.

1. Les tableaux

Exemple concret :

Imaginons :

```
int T[4] = {10, 20, 30, 40};
```

- taille d'un int : $t = 4$ octets
- adresse de départ : $d = 1000$
- Etant donnée que: Les éléments sont consécutifs, espacés de 4 octets chacun.

Alors :

Élément	Calcul d'adresse	Cases mémoire occupées
T[0]	$d + 0 \times 4 = 1000$	1000 → 1003
T[1]	$d + 1 \times 4 = 1004$	1004 → 1007
T[2]	$d + 2 \times 4 = 1008$	1008 → 1011
T[3]	$d + 3 \times 4 = 1012$	1012 → 1015

Tableau partiellement remplis

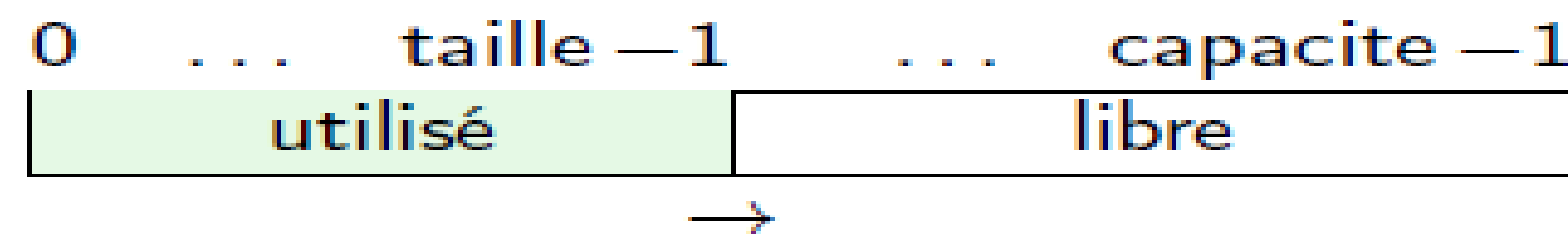
Retenir

Pour simuler un tableau de taille variable, on peut

- réserver une certaine quantité de mémoire appelée **capacité**,
- et ranger les valeurs **au début du tableau**.

Organisation des données (structure, classe) :

- tableau de taille capacité alloué
- éléments d'indice i pour $0 \leq i < \text{taille} \leq \text{capacité}$ initialisés



1. Les tableaux

💡 Principe

Simuler un tableau de taille variable à partir d'un tableau fixe :

- Réserver une certaine capacité (mémoire allouée)
- Utiliser seulement les premières cases selon la taille actuelle

⚙️ Organisation

Indice : 0 ... taille-1 →  utilisé

taille ... capacité-1 →  libre

🧠 Exemple 1

```
int tab[10]; // capacité = 10
```

```
int taille = 4;
```

```
tab[0]=12; tab[1]=7; tab[2]=5; tab[3]=9;
```

→ Tableau :

```
[ 12 | 7 | 5 | 9 |   |   |   |   |   |   ]
```

↑ utilisés (0-3) ↑ libres (4-9)

1. Les tableaux

Exemple 2 – Ajout d'un élément

tab[taille] = 15;

taille++;

→ Nouveau tableau :

[12 | 7 | 5 | 9 | 15 | | | |]

↑ utilisés (0–4)

À retenir

- capacité → espace total réservé
- taille → nombre d'éléments remplis
- Les valeurs sont rangées au début
- Permet de simuler un tableau dynamique simplement

Opérations de base

Hypothèses :

- tableau de taille capacite alloué
- éléments $0 \leq i < \text{taille} \leq \text{capacite}$ initialisés

Retenir (Opérations de base)

- *accès au premier élément* : $\Theta(1)$
- *accès à l'élément numéro i* : $\Theta(1)$
- *accès au dernier élément* : $\Theta(1)$
- *insertion/suppression d'un élément au début* : $\Theta(\text{taille})$
- *insert./suppr. d'un élt en position i* : $\Theta(\text{taille} - i) \subset O(\text{taille})$
- *insert./suppr. d'un élt à la fin* : $\Theta(1)$

Retenir — Les opérations de base et leur coût (complexité)

Opération	Signification	Complexité	Explication
Accès au premier élément	Lire <code>tab[0]</code>	$\Theta(1)$	Accès direct, instantané.
Accès à l'élément i	Lire <code>tab[i]</code>	$\Theta(1)$	En mémoire contiguë, on connaît directement l'adresse.
Accès au dernier élément	Lire <code>tab[taille - 1]</code>	$\Theta(1)$	Même logique, adresse connue immédiatement.
Insertion/Suppression au début	Ajouter/supprimer à <code>tab[0]</code>	$\Theta(\text{taille})$	Il faut décaler tous les éléments vers la droite/gauche.
Insertion/Suppression à la position i	Par ex. au milieu	$\Theta(\text{taille} - i) \subset O(\text{taille})$	Décalage partiel : on déplace les éléments à partir de i .
Insertion/Suppression à la fin	Ajouter/supprimer à <code>tab[taille-1]</code>	$\Theta(1)$	Pas besoin de décaler, opération immédiate.

Définition

- une liste linéaire = une suite d'éléments de même type repérés par leur emplacement,
- elle comporte n élément où n peut varier au cours de l'exécution,
- chaque élément est à une place donnée. Les places sont ordonnées et repérées par leur rang,
- la première place est appelé tête de la liste et la dernière "queue" de la liste
- on suppose qu'il y a une fonction successeur (succ) qui donne l'élément suivant pour chaque élément de la liste
- on va distinguer deux types de listes : accès direct sur un élément de rang i ou indirect de façon séquentielle à partir de la tête (la fonction succ)

3. Implémentation des listes chaînées à accès indirect en C/C++

```
typedef struct Place Place;
struct Place
{
    int val;
    struct Place *suiv;
};
typedef Place* Liste;
```

Ce code définit la structure d'une liste chaînée simple.

- **struct Place** représente un nœud contenant une valeur (val) et un pointeur vers le nœud suivant (suiv).
 - **typedef struct Place Place;** permet d'utiliser simplement le nom Place.
 - **typedef Place* Liste;** définit Liste comme un pointeur vers le premier nœud de la liste.
- 👉 En résumé : ce code prépare la création et la manipulation d'une liste chaînée d'entiers.

- Type : Liste, Place
- Utilise : Élément

Opérations

- 1 ListeVide : $\longrightarrow$ Liste
- 2 tête : Liste $\longrightarrow$ Place
- 3 Contenu : Place $\longrightarrow$ Élément
- 4 Cons : Liste $\times$ Élément $\longrightarrow$ Liste
- 5 Succ : Place $\longrightarrow$ Place
- 6 Premier : Liste $\longrightarrow$ Élément
- 7 Fin : Liste $\longrightarrow$ Liste

4.Types et opérations

N°	Opération	Type / Signature	Signification / Rôle	Exemple
1	ListeVide	$\rightarrow$ Liste	Crée une liste vide.	<code>L = ListeVide()</code> $\rightarrow$ <code>L = NULL</code>
2	tête	Liste $\rightarrow$ Place	Donne le premier nœud (place) de la liste.	<code>tête(L)</code> = premier nœud
3	Contenu	Place $\rightarrow$ Élément	Donne la valeur contenue dans un nœud.	<code>Contenu(p)</code> = <code>p->val</code>
4	Cons	Liste $\times$ Élément $\rightarrow$ Liste	Ajoute un élément en tête de la liste.	<code>Cons(L, 5)</code> $\rightarrow$ <code>[5] + L</code>
5	Succ	Place $\rightarrow$ Place	Donne le nœud suivant.	<code>Succ(p)</code> = <code>p->suiv</code>
6	Premier	Liste $\rightarrow$ Élément	Donne la valeur du premier élément.	<code>Premier(L)</code> = <code>L->val</code>
7	Fin	Liste $\rightarrow$ Liste	Donne la liste sans le premier élément (la suite).	<code>Fin(L)</code> = <code>L->suiv</code>

Axiomes

- ① tête(listevide), fin(listevide) et premier(listevide) ne sont pas définis,
- ② premier(cons(L,e)) = e,
- ③ premier(L) = contenu(tête(L))
- ④ fin(cons(L,e)) = L,
- ⑤ succ(tête(L)) = tête(fin(L))

5. QUELQUES Axiomes

Axiome 1

tête(listevide), fin(listevide) et premier(listevide) ne sont pas définis

→ Cela veut dire qu'on ne peut pas accéder au premier élément ou au reste d'une liste quand la liste est vide.

★ **Exemple :**

• liste = [] (liste vide)

Alors :

• tête(liste) → ✗ impossible

• fin(liste) → ✗ impossible

• premier(liste) → ✗ impossible

Axiome 2

premier(cons(L, e)) = e

→ cons(L, e) signifie : *ajouter l'élément e au début de la liste L.*

Donc le premier élément d'une liste construite par cons est e.

★ **Exemple :**

L = [3, 5, 8]

cons(L, 2) = [2, 3, 5, 8]

Donc :

premier(cons(L, 2)) = 2

Axiome 3

$\text{premier}(L) = \text{contenu}(\text{tête}(L))$

→ "tête(L)" est le premier **nœud** ou élément de la liste.

→ "contenu(tête(L))" est la **valeur de ce nœud**.

Autrement dit :

👉 $\text{premier}(L)$ = la valeur du premier élément.

★ **Exemple :**

$L = [4, 9, 1]$

$\text{tête}(L)$ = le nœud contenant la valeur 4

$\text{contenu}(\text{tête}(L)) = 4$

$\text{premier}(L) = 4$

Donc :

$\text{premier}(L) = \text{contenu}(\text{tête}(L)) = 4$

5. QUELQUES Axiomes

✓ Axiome 4

$$\text{fin}(\text{cons}(L,e)) = L$$

→ le "reste" d'une liste construite par cons est simplement la liste d'origine.

★ Exemple :

$$L = [10, 20, 30]$$

$$\text{cons}(L, 5) \rightarrow [5, 10, 20, 30]$$

Alors :

$$\text{fin}([5,10,20,30]) = [10,20,30] = L$$

✓ Axiome 5

$$\text{succ}(\text{tête}(L)) = \text{tête}(\text{fin}(L))$$

→ $\text{succ}(\text{tête}(L)) =$ le nœud qui vient après le premier

→ $\text{tête}(\text{fin}(L)) =$ le premier nœud de la liste sans son premier élément

Les deux sont identiques.

★ Exemple :

$$L = [7, 9, 11]$$

$$\text{tête}(L) = \text{nœud contenant } 7$$

$$\text{succ}(\text{tête}(L)) = \text{nœud contenant } 9$$

$$\text{fin}(L) = [9, 11]$$

$$\text{tête}(\text{fin}(L)) = \text{nœud contenant } 9$$

$$\text{Donc : } \text{succ}(\text{tête}(L)) = \text{tête}(\text{fin}(L)) = 9$$

ListeVide

- 1: ListeVide($L : \text{Liste}$)
- 2: $L \leftarrow \text{NULL}$ (ou retourner NULL)

Contenu

- 1: contenu(E : Élément, P : Place)
- 2: Entrée : P
- 3: Sortie : E
- 4: $E = P.val$ (ou retourner $P.val$)

SUCC

```
1: succ(P, Q : Place)
2: Entrée : P
3: Sortie : Q
4: Si P.suiv = NULL Alors erreur
5: Sinon Q ← *(P.suiv)
```


Fin

- 1: fin(L 1 :Liste, L2 : Liste)
- 2: Entrée : L1
- 3: Sortie : L2
- 4: Si L1 = NULL Alors erreur
- 5: Sinon L2 $\leftarrow$ L1->suiv

Définition d'une pile

◆ 1. Une pile = une liste où les éléments sont ajoutés et retirés d'un seul côté

On n'utilise qu'une **extrémité**, qu'on appelle :

➡ **le sommet de la pile** (top en anglais)

📌 **Analogie : une pile d'assiettes**

- On pose une assiette AU-DESSUS
- On retire une assiette AU-DESSUS

👉 On n'accède jamais à celle du bas directement.

Donc :

- **push(x)** → on ajoute x en haut
- **pop()** → on enlève ce qui est en haut

◆ 2. Une pile est LIFO : Last In First Out

➡ Le *dernier ajouté* est le *premier sorti*.

★ **Exemple :**

Si on fait :

1.push(10) → pile : [10]

2.push(5) → pile : [10, 5]

3.push(7) → pile : [10, 5, 7]

Si on fait pop(), on retire :

👉 **7**, parce que c'est le dernier entré.

✓ Applications des piles

◆ 1. Appel des fonctions récursives

Chaque appel de fonction est posé **sur la pile d'exécution**.

Exemple :

$f(3)$

→ $f(2)$

→ $f(1)$

→ $f(0)$

Quand $f(0)$ finit, on remonte :

• $f(1)$

• $f(2)$

• $f(3)$

➡ C'est exactement du LIFO.

Exemple

- ❶ $P \longrightarrow \text{pileVide}$
- ❷ `empiler(P,100)`
- ❸ `empiler(P,17)`
- ❹ `afficher(sommet(P))`
- ❺ `dépiler(P)`
- ❻ `afficher(sommet(P))`
- ❼ `dépiler(P)`
- ❽ `afficher(estVide(P))`

```
typedef struct Place Place;  
struct Place  
{  
    int val;  
    Place *prec;  
};  
typedef Place* Pile;
```

pileVide

- 1: pileVide($P : \text{Pile}$)
- 2: Sortie : P
- 3: $P \leftarrow \text{NULL}$ (ou retourner NULL)

Empiler

- 1: empiler($P : \text{pile}, E : \text{Élément}$)
- 2: Entrée : P, E
- 3: Sortie : P
- 4: $Q \longrightarrow \text{Allouer}(Q)$ (Q de type $*\text{Place}$)
- 5: $Q \rightarrow \text{val} \leftarrow E$
- 6: $Q \rightarrow \text{prec} \leftarrow P$
- 7: $P \leftarrow Q$ (ou retourner Q)

8. Implémentation piles

Comment fonctionne l'empilement (push) ?

 **Situation : la pile est vide**

`p = NULL`

 **1. On empile un second élément : 10**

On crée un nouveau nœud :

```
+-----+  
| val=10 |  
| prec= ? |  
+-----+
```

On le chaîne :

```
nouveau->prec = p; // p = NULL  
p = nouveau;       // p pointe maintenant ici
```

Résultat :

`p → [10 | NULL]`

8. Implémentation piles

Comment fonctionne l'empilement (push) ?

 **2. On empile un second élément : 20**

Créer un nouveau Place :

```
+-----+  
| val=20 |  
| prec= ? |  
+-----+
```

On règle son précédent :

```
nouveau->prec = p;    // p pointe sur [10 | NULL]
```

Maintenant :

```
nouveau -> [20 | * ]  
              |  
              V  
              [10 | NULL]
```

Puis on change le sommet :

```
p = nouveau;
```

Résultat final :

```
p -> [20 | -> ] -> [10 | NULL]
```

8. Implémentation piles

Comment fonctionne l'empilement (push) ?

 **3. On empile un troisième élément : 30**

Même principe :

```
+-----+  
| val=30 |  
| prec= p |  
+-----+
```

Après mise à jour :

```
p → [30 | → ] → [20 | → ] → [10 | NULL]
```

Dépiler

```
1: dépiler(P : pile)
2: Entrée : P
3: Sortie : P
4: SI  $P = NULL$  Alors erreur
5: Sinon
6:      $Q \longrightarrow P \rightarrow \text{prec}$ 
7:      $P \leftarrow Q$  (ou retourner Q)
```

Sommet

```
1: sommet(P : pile, E : Élément)
2: Entrée : P
3: Sortie : E
4: SI  $P = NULL$  Alors erreur
5: Sinon
6:      $E \longrightarrow P \rightarrow \text{val}$  (retourner  $P \rightarrow \text{val}$ )
```

8. Implémentation piles

🔵 Étape 1 : Vérifier si la pile est vide

Si $P = \text{NULL}$, cela veut dire qu'il n'y a aucun élément dans la pile.

Donc : impossible de dépiler $\rightarrow$ erreur.

🔵 Étape 2 : Déplacer le sommet

Supposons la pile :

$P \rightarrow [30 \mid \rightarrow] \rightarrow [20 \mid \rightarrow] \rightarrow [10 \mid \text{NULL}]$

Quand on dépile, on veut retirer **30**.

Ligne : $Q \leftarrow P \rightarrow \text{prec}$

- $P \rightarrow \text{prec}$ est l'élément juste en dessous dans la pile

- On le met dans Q

Donc maintenant :

$Q \rightarrow [20 \mid \rightarrow] \rightarrow [10 \mid \text{NULL}]$

Ligne : $P \leftarrow Q$

On met à jour le sommet :

$P \rightarrow [20 \mid \rightarrow] \rightarrow [10 \mid \text{NULL}]$

🎉 **Le 30 a été retiré.**

$\rightarrow$ La pile a simplement avancé d'un cran vers le bas.

```
SI P = NULL Alors erreur
```

```
Sinon
```

```
    Q ← P->prec
```

```
    P ← Q    (ou retourner Q)
```


8. Implémentation piles

Résumé du dépilage

Avant :

$P \rightarrow [30] \rightarrow [20] \rightarrow [10] \rightarrow \text{NULL}$

Après :

$P \rightarrow [20] \rightarrow [10] \rightarrow \text{NULL}$

8. Implémentation piles

Explication simple

Cette opération **ne modifie pas la pile**, elle lit juste le sommet.

● Étape 1 : Vérifier si la pile est vide

Même principe : si $P = \text{NULL}$, erreur $\rightarrow$ il n'y a pas de sommet.

● Étape 2 : Lire le sommet

$P \rightarrow \text{val}$ est la valeur du premier élément.

Exemple :

$P \rightarrow [20 \mid \rightarrow] \rightarrow [10 \mid \text{NULL}]$

Alors :

$E = 20$

La pile ne change pas.

```
SI P = NULL Alors erreur
```

```
Sinon
```

```
    E ← P->val    (retourner P->val)
```

7. Implémentation piles

Résumé du sommet

Avant :

$P \rightarrow [20] \rightarrow [10] \rightarrow \text{NULL}$

Sommet(P) retourne :

20

Après : (la pile ne bouge pas)

$P \rightarrow [20] \rightarrow [10] \rightarrow \text{NULL}$

```
SI P = NULL Alors erreur
```

```
Sinon
```

```
    E ← P->val    (retourner P->val)
```

EstVide

- 1: estVide(P : pile, B : Booléen)
- 2: Entrée : P
- 3: Sortie : B
- 4: $B \leftarrow (P == \text{NULL})$ (retourner $(P == \text{NULL})$)

8. Implémentation piles

```
estVide(P : pile, B : Booléen)  
Entrée : P  
Sortie : B  
B ← (P == NULL)    (retourner (P == NULL))
```

- L'algorithme *estVide* teste si le pointeur de pile *P* vaut NULL.
- S'il est NULL, la pile est vide *B* = Vrai;
- sinon, elle contient au moins un élément, *B*=Faux.

Définition

- une file est une liste pour laquelle les éléments sont ajoutés à une extrémité et retiré d'une autre,
- elles portent le nom de First In First Out (FIFO) : premier entré, premier sorti,
- les opérations définies sur les files sont : accéder au premier élément, ajouter un élément, retirer le premier élément, tester si la file est vide.

Définition d'une file (Queue)

✓ 1. Une file est une liste spéciale

- Dans une file, **on ajoute les éléments d'un côté** (à l'arrière),
- Et **on retire les éléments de l'autre côté** (à l'avant).

📌 Exemple imagé :

Imagine une file d'attente à la banque :

- Le nouveau client arrive à **la fin de la file**.
- Le client servi est **celui qui était en tête**, donc le premier arrivé.

✓ 2. Fonctionnement : FIFO

FIFO = **First In, First Out**

En français : **Premier entré, premier sorti.**

Cela veut dire que l'élément ajouté en premier sera celui qui sortira en premier.

 Exemple :

On ajoute : $A \rightarrow B \rightarrow C$ // A, le suivant B et le suivant C

Si on retire un élément, on retire **A** en premier.

✓ 3. Les opérations possibles sur une file

➤ Accéder au premier élément

Sans le retirer, juste voir ce qu'il y a en tête.

➤ Ajouter un élément

On ajoute toujours à la fin de la file.

➤ Retirer le premier élément

On enlève celui qui est à l'avant, le plus ancien.

➤ Tester si la file est vide

Vérifier s'il n'y a aucun élément.

Implémentation avec pointeurs

```
typedef struct Place Place;
struct Place
{
    int val;
    Place *suiv;

};
typedef struct File File;
struct File
{
    Place *debut
    Place *fin;

};
```

1. La structure Place : 1 personne dans la file

```
struct Place
{
    int val;
    Place *suiv;
};
```

Imagine une file d'attente avec des personnes.

Une Place = une personne.

- `int val;` = le numéro de la personne
- `Place *suiv;` = un doigt qui montre la personne suivante dans la file

Exemple :

`[Personne 10] → [Personne 5] → [Personne 42] → NULL`

- Chaque bloc contient une valeur (**val**) et un pointeur vers le bloc suivant (**suiv**).

Donc **Place** = un maillon de la liste chaînée.

2. La structure File : la file d'attente

struct File

{ Place *debut;

Place *fin;

};

File représente toute la file.

Elle a **deux pointeurs** :

debut → la première personne

fin → la dernière personne

Exemple :

début

fin



[10] → [5] → [42] → NULL

Conclusion:

Pour **retirer** quelqu'un → on enlève au début

Pour **ajouter** quelqu'un → on ajoute à la fin

C'est très rapide.

ÉLÉMENT	RÔLE
Place	un élément de la file
Place.val	la valeur
Place.suiv	lien vers l'élément suivant
File	la structure qui contient toute la file
File.debut	premier élément
File.fin	dernier élément

Enfiler (ajout d'un élément à la fin)

```
1: enfiler(F : File, E : Élément)
2:   Entrée : F, E
3:   Sortie : F
4:   Allouer(Q)
5:   Q->val ← E
6:   Q->suiv ← NULL
7:   Si F.debut = NULL Alors
8:     F.debut ← Q
9:     F.fin ← Q
10:  Sinon
11:    (F.fin)->suiv ← Q
12:    F.fin ← Q
```

Explication de l'algorithme *Enfiler(F, E)* (ajout en fin)

Une **file** (queue) fonctionne selon le principe **FIFO** (First In – First Out).

On **ajoute à la fin**, on **retire au début**.

La file contient deux pointeurs :

- `F.debut` : pointe vers le premier élément
- `F.fin` : pointe vers le dernier élément

1–3 : Entête

Ces lignes décrivent :

- l'opération : `enfiler(F, E)`
- les entrées : la file `F` et l'élément `E`
- la sortie : la file mise à jour

10. Implémentation des files

Allouer(Q)

On crée un **nouveau nœud** Q dans la mémoire.

Q->val ← E

On met dans le nœud Q la valeur à ajouter.

Q->suiv ← NULL

Comme ce nœud sera à la fin, **il n'a pas de suivant**. Donc son pointeur suivant = NULL.

Si F.debut = NULL Alors

On teste si **la file est vide**.

Une file est vide si debut = NULL (donc aucun élément).

Cas file vide

Si la file est vide :

$F.\text{debut} \leftarrow Q$

Le nouveau nœud devient **le premier élément**.

$F.\text{fin} \leftarrow Q$

Il devient aussi **le dernier** (puisque'il est seul).

Sinon (file non vide)

$(F.\text{fin}) \rightarrow \text{suiv} \leftarrow Q$

On relie l'ancien dernier élément au nouveau :

$\text{dernier} \rightarrow Q$

$F.\text{fin} \leftarrow Q$

On met à jour fin : maintenant le dernier élément est Q.

Résumé visuel

Si la file est vide :

[Q] (debut = Q, fin = Q)

Sinon :

Avant :

debut → [A] → [B] → [C] (fin = C)

Après enfiler(E) :

debut → [A] → [B] → [C] → [Q] (fin = Q)

Premier (récupération de la valeur du premier élément)

```
1: Premier(F : File, E : Elément)
2:   Entrée : F
3:   Sortie : E
4:   Si F.debut = NULL Alors
5:     Erreur
6:   Sinon
7:     E ← (F.debut)->val
```

Défiler (suppression du premier élément)

```
1: defiler(F : File)
2:   Entrée : F
3:   Sortie : F
4:   Si F.debut = NULL Alors Erreur
5:   Sinon
6:     F.debut ← (F.debut)->suiv
7:   Si F.debut = NULL Alors
8:     F.fin ← NULL
```

1. Premier(F, E)

👉 *Récupération (lecture) de la valeur du **premier élément** de la file, sans le supprimer.*

🔍 Étapes expliquées

1. On reçoit la file **F** et une variable **E** (qui contiendra la valeur lue).

2. Si **F.debut = NULL**, cela signifie que la file est **vide** → on ne peut rien lire → **Erreur**.

3. Sinon, la file contient au moins un élément →

➤ On met dans **E** la valeur du premier nœud :

$E \leftarrow (F.debut) \rightarrow val$

🧠 Idée générale

C'est juste une **lecture**, pas une suppression.

On lit la valeur du premier élément, mais la file reste intacte.

12. defiler(F)

👉 *Suppression du premier élément de la file.*

🔍 Étapes expliquées

On reçoit la file F (en entrée et en sortie, car elle sera modifiée).

Si F.debut = NULL, la file est vide → impossible de supprimer → **Erreur**.

Sinon :

➤ On avance le début de la file :

$F.debut \leftarrow (F.debut) \rightarrow suiv$

Si après cela F.debut devient NULL (cela arrive si on a supprimé le **dernier élément**) :

➤ Il n'y a plus aucun élément → on met aussi **F.fin = NULL**

 Idée générale

On **retire** le premier élément :

si la file devient vide, on met début et fin à NULL

sinon, juste début avance

Opération	Action	Modifie la file ?	Erreurs possibles
Premier()	Lit la valeur du premier élément	 NON	File vide
Défiler()	Supprime le premier élément	 OUI	File vide